Simplified Message Transformation for Optimization of Message Processing in 3G-324M Control Protocol*

Man-Ching Yuen [1], Ji Shen [2], Weijia Jia [3], and Bo Han [4]

Department of Computer Science, City University of Hong Kong,
83 Tat chee Avenue, Kowloon Tong, Kowloon, Hong Kong, China
[1] connie.yuen@alumni.cityu.edu.hk
[2] ji.shen@student.cityu.edu.hk
[3] itjia@cityu.edu.hk
[4] Bo.Han@student.cityu.edu.hk

Abstract. 3G-324M is a multimedia transmission protocol designed for 3G communication environment. Meanwhile H.245 standard is a control protocol in 3G-324M and gives specific descriptions about terminal information messages in H.245 control channel as well as the procedures using them. The message syntax is defined using an external data representation standard called Abstract Syntax Notation One (ASN.1). For transmission, ASN.1 formatted data is transformed into bit-stream based on an ASN.1 encoding standard called Packed Encoding Rules (PER). In order to meet the requirement of high speed data transfer in 3G communication, it is important to design the procedure of message processing as simple as possible. In this paper, we propose *Single-step Direct Message Transformation (SDMT)* for the optimization of tree-structured message processing in H.245 module. By testing in realistic environments in some China industries, performance evaluation shows that code redundancies in terms of file size and code size are reduced significantly.

1 Introduction

With wider bandwidth of third-generation networks (3G) and increasing number of multimedia service categories, the mobile communication market has grown at an explosive rate in recent years, especially 3G is launching in different places in the world. 3G wireless multimedia communications are particularly referred to as International Mobile Telecommunications 2000 (IMT-2000) that has been deployed and developed substantially. 3G-324M [1] is a standard umbrella protocol for supporting multimedia transmission using 3G technologies. In 3G communication environments, it consists of a signaling channel which is used for the exchange of capabilities and opening of video, audio and data channels between two different phones. The signaling channel is defined by H.245 protocol [2].

* The work is supported by Research Grant Council (RGC) Hong Kong, SAR China, under grant nos.: CityU 1055/00E and CityU 1039/02E and CityU Strategic grant nos. 7001709 and 7001587 and This paper is sponsored by 973 National Basic Research Program, Minister of Science and Technology of China under Grant No. 2003CB317003.

X. Lu and W. Zhao (Eds.): ICCNMC 2005, LNCS 3619, pp. 64 – 73, 2005.

H.245 standard has been defined to be independent of the underlying transport mechanism, but is intended to be used with a reliable transport layer, which provides guaranteed delivery of correct data. H.245 specifies syntax and semantics of messages as well as the procedures for in-band negotiation at the start of or during communication. The message syntax is defined using Abstract Syntax Notation One (ASN.1) [3]. ASN.1 is a specification language for describing structured information using in communication protocols and allows a protocol designer to define parameters in Protocol Data Units (PDU) without concerning how they are encoded for transmission. The definition of data types in ASN.1 can be grouped into two categories: primitive type and constructive type. If the data value contains other data values, then it is defined as a constructive type, otherwise it is a primitive type. There are over 20 primitive data types in ASN.1 including BOOLEAN, INTEGER, ENUMBERATED, REAL, BIT STRING, OCTET STRING, NULL, ANY and OBJECT IDENTIFIER. Examples of constructive data types in ASN.1 are SET, SEQUENCE, SET OF, SEQUENCE OF and CHOICE. For transmission of messages in H.245 control channel, ASN.1 formatted data is transformed into bit-stream based on an ASN.1 encoding standard called Packed Encoding Rules (PER) [4]. PER is one of derivatives of Basic Encoding Rules (BER) [5] and especially designed for high speed data transfer. PER provides a much more compact encoding than BER and tries to represent data units using the minimum number of bits.

A BER encoding is highly structured [5] and comprised of a sequence of octets. In BER, ASN.1 uses Tag-Length-Value (TLV) format for encoding the data types. The TAG field consists of a tag which is uniquely associated with a specific ASN.1 data type. The LENGTH field indicates the length of the contents encoded in case of definite length encoding, while indefinite length encoding uses an end-of-contents (EOC) indicator to delimit the contents. The VALUE field contains either a value of a primitive type or values of different component types of a constructive type.

PER is not a TLV style of encoding, so tags are not encoded at all. Only data of some primitive types will have their length encoded. Data of constructive types do not have their length encoded explicitly; instead they rely on their components to define their length. The compactness of PER encodings requires that the decoder knows the complete original abstract syntax of the data structure to be decoded. In other words, PER encodings are not self-defining, thus less flexible than BER encodings. As a result, the implementation of PER is much complicated than BER.

With the control of H.245 module during a communication session, messages are generated and encoded into binary bits streams in ASN.1 syntax based on PER. After that, the encoded bits streams are delivered to transport layer to send to the peer communicators. In both initialization and the communication stages, H.245 module is first invoked, and then messages are generated and sent dynamically during the runtime. As a result, message processing of encoding and decoding may be invoked many times flexibly, and its efficiency will affect the whole performance of H.245 module.

Based on the above observations, in this paper, we propose a *Single-step Direct Message Transformation (SDMT)* for the optimization of tree-structured message processing in H.245 module. It increases the difficulty to design SDMT for PER which is a complicated encoding scheme. Our implementation has been tested in a realistic heterogeneous 3G communication environment in some China industries. It shows that the scheme of SDMT has lower code redundancy and higher efficiency because of the simplified encoding and decoding routines.

The rest of the paper is organized as follows. Section 2 describes the common tree-structured implementations of message processing in H.245 module. Section 3 presents our proposed Single-step Direct Message Transformation (SDMT) for message processing in H.245 module. Its implementation and performance evaluation are presented in Section 4. Section 5 concludes the paper.

2 Tree-Structured Message Processing in H.245 Module

To provide guaranteed delivery of correct data, H.245 specifies syntax and semantics of terminal information messages in H.245 control channel as well as the procedures for in-band negotiation at the start or during the communication. The messages cover receiving and transmitting capabilities as well as mode preference, logical channel signaling and control. In H.245 module, Signaling Entity (SE) is referred to as a procedure that is responsible for special functions. It is designed as state machine and changes its current state upon reaction to an event occurrence.

In H.245 module, messages are defined in tree-like structure. H.245 defines a general message type *MultimediaSystemControlMessage* (MSCM). Four types of special messages are further defined in MSCM as *request*, *response*, *command* and *indication*. A *request* message results in a specific action and requires an immediate response. A *response* message responds to a request message. A *command* message requires an action but no explicit response. An *indication* message contains information that does not require action or response. Messages with various types are transformed into MSCM for uniform processing and parameters in MSCM are set to distinguish different types of messages.

Each of the four types of special messages has a number of its own subtypes, and is further defined as one of them. The number of subtypes of request, response, command and indication is 16, 25, 13 and 24 respectively. A message defined with subtype consists of values of a number of elements of ASN.1 notation. These elements may be of primitive type or constructive type. As mentioned in Section 1, a constructive type is defined by a number of primitive types and constructive types. An illustrative example is shown in the following. For MasterSlaveDetermination type, it is defined as a RequestMessage in the first level of message definition, and then the definition is refined as a MasterSlaveDeterminationMessage in the second level of message definition. Finally, the definition of all elements of MasterSlaveDeterminationMessage is declared in the third level of message definition.

Message Definition in H.245 Specification

```
Level 1 Definition (Definition of MultimediaSystemControlMessage)
MultimediaSystemControlMessage       ::=CHOICE
{
    request   RequestMessage,
    response  ResponseMessage,
    command   CommandMessage,
    indicationIndicationMessage,
    ...
}
```

Level 2 Definition (Definition of RequestMessage of MultimediaSystemControlMessage)

```
RequestMessage::=CHOICE
{
    nonStandard NonStandardMessage,
    masterSlaveDetermination MasterSlaveDetermination,
    terminalCapabilitySet TerminalCapabilitySet,
    openLogicalChannel OpenLogicalChannel,
    closeLogicalChannel CloseLogicalChannel,
    requestChannelClose RequestChannelClose,
    multiplexEntrySend MultiplexEntrySend,
    requestMultiplexEntry RequestMultiplexEntry,
    requestMode RequestMode,
    roundTripDelayRequest RoundTripDelayRequest,
    maintenanceLoopRequest MaintenanceLoopRequest,
    ...,
    communicationModeRequest CommunicationModeRequest,
    conferenceRequestConferenceRequest,
    multilinkRequest MultilinkRequest,
    logicalChannelRateRequest LogicalChannelRateRequest,
    genericRequest GenericMessage
}
```

Level 3 Definition (Definition of MasterSlaveDeterminationMessage of RequestMessage of MultimediaSystemControlMessage)

```
MasterSlaveDetermination ::=SEQUENCE
{
    terminalType INTEGER (0..255),
    statusDeterminationNumber INTEGER (0..16777215),
    ...
}
```

Since H.245 defines messages in tree-structure, following the specification in H.245, the procedure of message processing is also in tree-structure. In tree-structured message processing approach, it contains numerous encoding/decoding routines and parsing routines. They are in parallel to corresponding data representations at specific message definition levels in H.245 protocol. Hence, the top-level encoding/decoding routines correspond to the definition of messages, and the bottom-level contains the encoding/decoding routines of each ASN.1 given data type. The encoding/decoding routines provide translations of ASN.1 data between abstract type and transfer type, while the parsing routines are designed for classification of elements of a message definition level in H.245 protocol.

Fig. 1 shows the flowchart of tree-structured message processing in H.245 module. In order to process messages in H.245 module, the top-level parsing routine calls the lower level parsing routines, and the parsing routines at different levels set values to classify their element types which are stored as intermediate data representations in buffer. The process of parsing continues and is complete when the lowest level of parsing is called. Once messages are classified, by using the intermediate data stored in buffer, the encoding process starts to transform the terminal information message

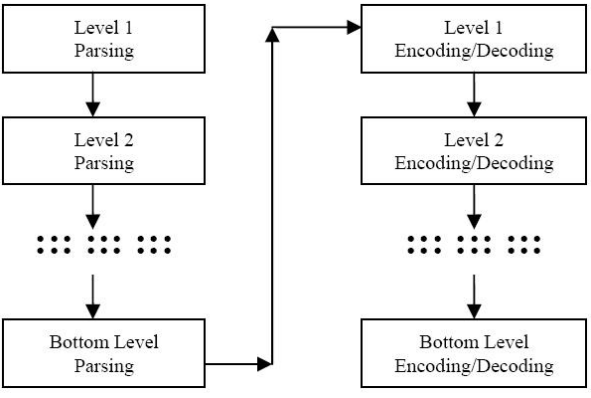

Fig. 1. Flowchart of Tree-structured message processing in H.245 module

into bit-stream. The top-level encoding routine calls the lower level encoding routines and the encoding routines at different levels set values to the corresponding items in a message. The processes go ahead until the messages are encoded into a bit-stream eventually. The similar work is done for the decoding process.

The tree-structure approach works efficiently in dealing with maintenance. However, as parsing process has to be completed before encoding/decoding process starts, the tree-structured definition in H.245 protocol has to be considered in parsing process and encoding/decoding process separately. The lengthy and complicated encoding rules are executed step by step along the tree-structure for twice. Note that the lower the level of definition of message is, the higher the complexity of message definition is. Thus, the message processing is still complex especially for a complicated encoding scheme such as PER and it results in a high code redundancy.

For existing implementation, codes of encoding and decoding are usually automatically generated by ASN.1 compilers. The ASN.1 compiler proposed in [6] called CASN.1 translates data without converting data into an intermediate form but is only based on the fundamental encoding scheme BER, while PER for high speed data transfer in 3G is much complicated and not suitable to use. Based on these observations, in this paper, we propose Single-step Direct Message Transformation (SDMT) in H.245 module. The methodology of SDMT will be described in detail in Section 3.

3 Single-Step Direct Message Processing (SDMT) in H.245 Module

For data transfer, terminal information messages have to be processed in H.245 module before transmission. In H.245 module, all messages are expected to be parsed into MSCM and then encoded by PER. Thus messages received from peer terminals should be parsed and decoded for further processing. The implementation of parsing and encoding/decoding involves some similar memory access and bitwise logical operations. To better utilize the limited resources of a terminal, we propose a simple and efficient approach on implementation of message processing in H.245 module

called *Single-step Direct Message Transformation* (SDMT), in which some procedures in parsing and encoding/decoding are compressed and integrated.

The differences of processing flow between tree-structured implementation and our single-step implementation are described as follows. In the tree-structured implementation, to transfer data, we need to parse a semantic terminal information message into MSCM in accordance with tree-structure specification, and then MSCM is encoded into a bit-stream by PER based on the tree-structured specification again. As the tree-structure specification has to be referred twice, the recursive callings of descend functions are executed twice, one for parsing and the other for encoding. Besides, the data received by peer terminals is also parsed and decoded for further processing. In our implementation, in order to simplify the process, we have combined the parsing and encoding procedures into one procedure by directly transforming the message (i.e., the leaves of the tree) into a bit-stream. Based on the same idea, the decoding procedures are also combined with the corresponding parsing procedures. As the message is not reduced into MSCM as defined in H.245 specification, the complexity of message transformation between semantic messages of ASN.1 notation and encoded messages with use of PER is greatly decreased.

Fig. 2 shows the flowchart of Single-step Direct Message Processing (SDMP) in H.245 module. The top-level encoding routine calls the lower level encoding routines. Unlike tree-structured message processing, SDMP combines the parsing and encoding routine of the same level into one integrated encoding routine. Thus the encoding routine at each level calls the corresponding parsing routine, classifies the structure of message at the view of the level, sets values to the corresponding items and then encodes them accordingly. The processes go ahead until messages are encoded into a bit-stream eventually. The similar work is done for the decoding process.

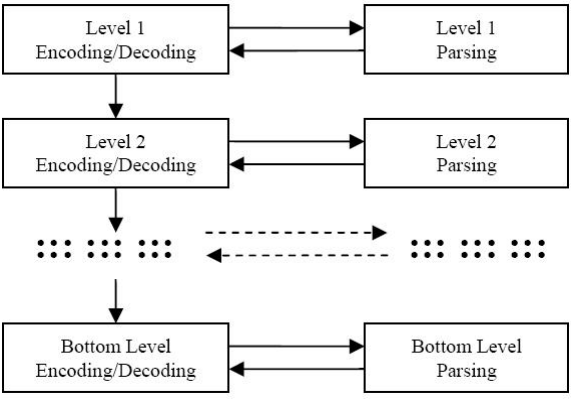

Fig. 2. Flowchart of Single-step Direct Message Processing in H.245 module

Our implementation has two main characteristics: (1) *Parsing procedures execute within the encoding/decoding procedures at each level.* It differs from tree-structured approach which all parsing procedures have to be completed before the encoding procedures start. The reason for the feasibility of our approach is described as follows.

In H.245 module, information messages are defined and presented in the form of a number of levels for increasing readability when implementing the protocol. SDMP translates data values directly between the ASN.1 formatted data structures and the PER transfer syntax thus eliminating the overhead in converting the data into an intermediate stage. In this way, it greatly reduces the encoding/decoding codes with same maintainability and simplifies their complexity of encoding/decoding operations.

(2) For high-level messages containing a number of ASN.1 data values, in our simplified implementation concepts, a high-level message and its ASN.1 data values can be viewed as a linked list and nodes connected by the linked list respectively. It makes the coding in encoding/decoding procedures still to be readable and maintainable. In abstract concept, each information message contains a number of elements that are ASN.1 data structure. An element in a message is taken as a node and the nodes in a message are linked together into a list that represents the message. It gives a quick mapping to the syntax of MSCM. All messages in a communication sessions are further linked together into a global list. The global list refers to the encoded bit stream, which is the output of bottom level encoding routine. In [4], PER rules are defined according to different data types, thus we have implemented the explicit encoding/decoding functions for each data type accordingly. When encoding/decoding a message, the nodes in the message list are encoded one by one using corresponding PER data type encoding functions. This approach is still easy to manage, and we have packed the procedures into our final implementation. As a result, based on this approach, the PER codec in our implementation is simplified without increasing the workload of encoding/decoding operations or modifying the syntax and semantics of the messages.

Without encoding tags indicating types of encoded data, PER is less flexible than BER. Thus the design of ASN.1 compiler with the use of PER is much complex than BER, and it is necessary to have a simpler design approach for PER codec compared with BER codec. Our PER codec implementation is simplified based on Single-step Direct Message Processing (SDMP) design approach compared with that of others. In SDMP, each encoded ASN.1 data structure can be self-descriptive and used in high-level encodings directly. In this way, without implementing the intermediate syntax in our PER codec, specific ASN.1 syntax specification information still can be included in encoded bit stream, thus enough information is obtained for decoding process in PER after data transfer. Moreover, our H.245 module is much simple in design and implementation.

4 Our Implementation

Our PER codec is written in programming language C. We select C language as our target language, because translation for most data types between ASN.1 and C can be achieved simply by direct mapping. As ASN.1 contains a richer set of types, some ASN.1 types require programmer defined C structures for the translation. However, all the ASN.1 types can be simply represented by using C structures. The major part of the ASN.1 standard is presently implemented in our PER codec.

In the implementation of our PER codec, the C files contain encoding and decoding routines for data types of both primitive type and constructive type. Note that in

SDMP, each encoded ASN.1 data structure can be self-descriptive and used in high-level encodings directly. We present the implementation issues in our Single-step Direct Message Processing (SDMP) in H.245 module in Section 4.1 and evaluate the performance of our approach comparing with the tree-structure message processing approach in Section 4.2.

4.1 Message Processing

In our PER codec, without implementing the intermediate syntax, specific ASN.1 syntax specification information still can be included in encoded bit stream, thus enough information is obtained for decoding process in PER after data transfer. Moreover, our H.245 module is much simple in design and implementation. In this part, the implementation of message processing of our PER codec is presented in detail. As the definition of MasterSlaveDetermination message is shown in Section 2, it is also used as an example to demonstrate our implementation. The following shows the pseudo code for encoding of MasterSlaveDetermination message.

Pseudo code for encoding of MasterSlaveDetermination message

```
EncodedBits* h245_encode_MasterSlaveDetermination(BYTE *bit_stream, int
*pos, int terminalType, int statusDeterminationNumber)
{
   EncodedBits *head,*tail,*newBits
   enc_init(&head,&tail))
    /* encode the choice in MultimediaSystemControlMessage */
   newBits = enc_choice(3,RequestMessage_chosen,4)
   encode_append_bits(head,&tail,newBits)
    /* encode the choice in RequestMessage */
   newBits = enc_choice(10,MasterSlaveDetermination_chosen,16)
   encode_append_bits(head,&tail,newBits)
    /* encode the extension marker in MasterSlaveDetermination message */
   newBits = enc_seq(0,1)
   encode_append_bits(head,&tail,newBits)
    /*** start encoding content of MasterSlaveDetermination message ***/
    /* encode the terminalType */
   newBits = enc_integer(0,255,terminalType)
   encode_append_bits(head,&tail,newBits)
   /* encode the statusDeterminationNumber */
   newBits = enc_integer(0,16777215,statusDeterminationNumber)
   encode_append_bits(head,&tail,newBits)
    /*** end of encoding content of MasterSlaveDetermination message ***/
    /* concatenate the encoded bit stream */
   enc_concatenate(head,&tail)
    return head
}
```

There are four input parameters: (1) *bit_stream* is a pointer pointing to the input bit stream data; (2) *pos* indicates the bit position of the first byte of the bit_stream; (3) *terminalType* is one of the two elements of MasterSlaveDetermination message and it is an integer between 0 to 255; (4) *statusDeterminationNumber* is another element of MasterSlaveDetermination message and it is an integer between 0 to 16777215. After

initialization of a pointer of bit stream, the message is encoded based on the first level message definition, which is MultimediaSystemControlMessage. It is CHOICE type with an extension marker after index 3 and has 4 elements totally. MasterSlaveDetermination message is defined as a RequestMessage in MultimediaSystemControlMessage definition, while *Request_chosen* is a constant representing the index of RequestMessage in the MultimediaSystemControlMessage definition. Note that, the encoded bits resulted from the encoding of each level message definition are appended to a bit stream. In next step, message encoding is based on the second level message definition, Request message. Request message is a CHOICE type with an extension marker after index 10 and has 16 elements totally. *MasterSlaveDetermination_chosen* is a constant representing the index of MasterSlaveDetermination message in the RequestMessage definition.

MasterSlaveDetermination message is of SEQUENCE type. For encoding of SEQUENCE type, it has an extension marker but without extension part, so that the first input parameter is 0. The bit map is 1 because the message has two elements and the index of the last element is 1. Note that the indexing starts from 0. The two INTEGER elements of the MasterSlaveDetermination message are then encoded and appended to the bit stream. Finally, the bit stream is concatenated in the octet string (a multiple of 8 bits) if aligned PER encoding is used.

The decoding procedure of MasterSlaveDetermination message is just the reverse of the encoding process of the message. Firstly the MasterSlaveDetermination message is decoded based on the first level message definition MultimediaSystemControlMessage, and the index of RequestMessage in the CHOICE is resulted. Then, the message is decoded based on the second level message definition RequestMessage, it results the index of MasterSlaveDetermination message in the CHOICE. Finally, the decoding of the bottom level message definition (decoding of the ASN.1 basic data types) carries out, and all the elements of MasterSlaveDetermination message, *terminalType* and *statusDeterminationNumber*, are decoded.

4.2 Performance Evaluation

In this section, we evaluate the performance of our Single-step Direct Message Processing (SDMP) approach comparing with the tree-structure message processing approach. Our implementation has been tested in a realistic heterogeneous 3G communication environment in some China industries. Applying Single-step Direct Message Transformation (SDMT) in H.245 module, the intermediate processes are skipped and the complexity of the overall process in H.245 is expected to be decreased. Two measurement criteria are used to evaluate the performance for code redundancies called *file size* and *code size*. The file size is the size of files containing the source code, while the code size is the size of files containing object code originated from compilation. Table 1 shows the performance comparison on H.245 module between two implementation approaches, and they are traditional tree-structured message processing and our Single-step Direct Message Transformation (SDMT).

Table 1. Performance Comparison on H.245 module between two implementation approaches

	File Size (Source code)	Code Size (Object generated from compilation)
Traditional approach	434 KB	261 KB
Our proposed SDMT	306 KB	221 KB
Reduction Rate of SDMT	**29.5%**	**15.3%**

The file and code sizes of the implementation of traditional design as the PER specification are 434 KB and 261 KB, respectively. In the implementation of Single-step Direct Message Transformation (SDMT), the file size is 306 KB and the code size is 221 KB. As a result, the reduction rates in file size and code size of our approach are 29.5% and 15.3% respectively. It shows a significantly reduction in code redundancies, which is very important for a mobile terminal with limited memory.

5 Conclusion

In this paper, we proposed *Single-step Direct Message Transformation (SDMT)* for the optimization of tree-structured message processing in H.245 module, which is the control module defined in an umbrella protocol 3G-324M for supporting multimedia communication in 3G environment. SDMT simplifies the encoding/decoding routine in H.245 message processing even using PER codec which can give high compression rate but also high implementation complexity. Performance evaluation shows that code redundancies in terms of file size and code size are reduced significantly.

References

1. ITU-T H.324, Terminal for low bit-rate multimedia communication, Int'l Telecommunication Union, 2002.
2. ITU-T H.245, Control protocol for multimedia communication, Int'l Telecommunication Union, 2003.
3. ITU-T ISO/IEC IS 8824: 1987, Information processing systems – Open Systems Interconnection – Specification of Abstract Syntax Notation One (ASN.1), Int'l Telecommunication Union, 1987.
4. ITU-T ISO/IEC IS 8825-2: 1995, Information Technology – ASN.1 encoding rules: Specification of Packed Encoding Rules (PER), Int'l Telecommunication Union, 1995.
5. ITU-T ISO/IEC IS 8825: 1987, Information processing systems – Open Systems Interconnection – Specification of Basic Encoding Rules for Abstract Syntax Notation ONE (ASN.1), Int'l Telecommunication Union, 1987.
6. G.W. Neufeld, Y. Yang, "The Design and Implementation of an ASN.1-C Compiler," IEEE Transactions on Software Engineering, vol. 16, no. 10, Oct. 1990, pp. 1209-1220.